

Q. No: 7
Date: 18/7/2026

CONCEPT MAP: RECURRENCE RELATION $\rightarrow$ RECURSION TREE

LEVEL COST $\rightarrow$ TOTAL COST $\rightarrow$ COMPLEXITY

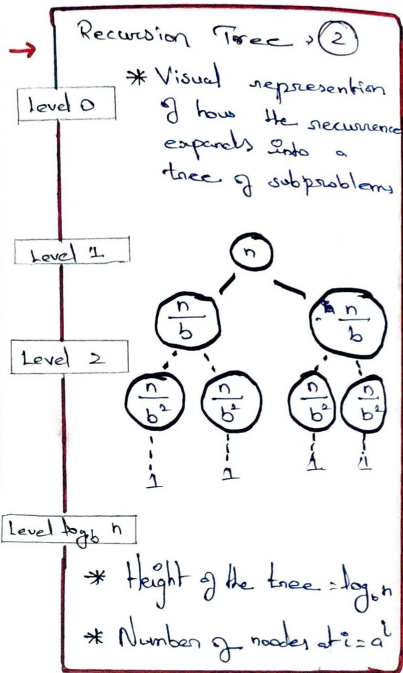
Name:- A. Bhavani
Course no:- CSPO606
Reg No:- 192524
325

1. Recurrence Relation
* Mathematical equation that defines a problem in terms of smaller subproblems

General form (Divide and Conq)
 $T(n) = aT(n/b) + f(n)$

$a \rightarrow$ number of subproblems
 $\frac{n}{b} \rightarrow$ size of each subproblem
 $f(n) \rightarrow$ non-recursive work

Base Case
 $T(n) = \Theta(1)$ for $n \leq n_0$



3. Level Cost
Cost incurred to solve all subproblems at the same level.

At level i :
of nodes $= a^i$
Size of each $= n/b^i$
Cost of per node $= f(n/b^i)$
Cost of level $i = a^i \cdot f(n/b^i)$

$T(n) = 2T(n/2) + n$
At level i : cost $=$
 $2^i \cdot \frac{n}{2^i} = n$ (Each level)

Total cost (4)
Sum of cost of all levels from root of leaves

Total cost
 $\Rightarrow \sum_{i=0}^{\log_b n-1} a^i \cdot f(n/b^i)$
+ (Num of leaves) $\times \Theta(1)$

Eg $T(n) = 2T(n/2) + n$
Total cost $= \sum_{i=0}^{\log_2 n-1} n + n$
 $= \Theta(n \log n)$

5. Complexity
Growth rate of total cost as a function of input size n .
Expresses using Big-O notation

Result:
 $T(n) = 2T(n/2) + n$
 $\Rightarrow \Theta(n \log n)$

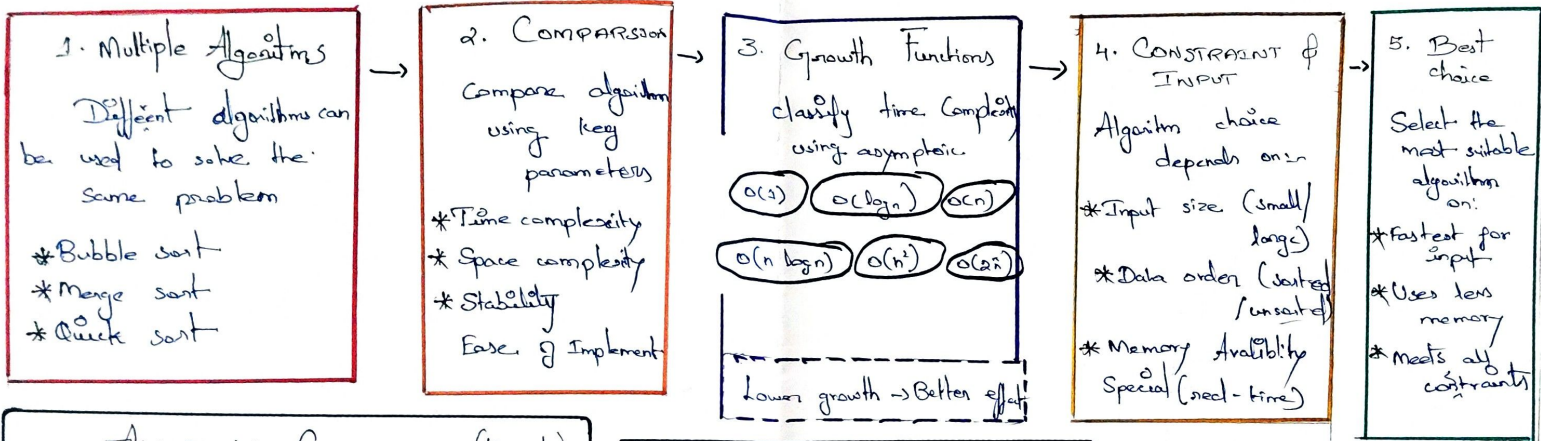
* Visualize recursion clearly

WHY RECURSION TREE METHOD IS EFFECTIVE?
* Breaks problems into level (hierarchy)
* Helps derive tight bounds and Big-O efficiently

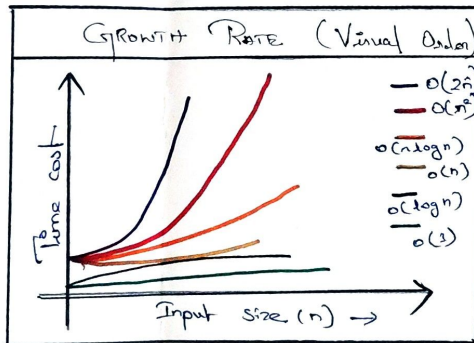
Q.No: 47
Date: 18/7/2026

CONCEPT MAP: MULTIPLE ALGORITHMS → COMPARISON → GROWTH FUNCTION → BEST CHOICE.

Name:- H. Bhanani
Course:- CSA0606
Reg No:- 192524385



ALGORITHM COMPARISON (Example)					
Algorithm	Best	Average	Worst	Space	Stability
Bubble sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	(X)
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	(✓)
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	(X)
Heap sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	(X)
Insertion sort	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(1)$	(✓)



How To choose?

→ For small input: Any simple algorithm may work.

→ For large input: Prefer $O(n \log n)$ or better.

→ If memory is limited: Choose $O(1)$ or $O(\log n)$ space.

★ Predict Performance

★ Fair Comparison

★ Optimal selection

★ Efficiency & scalability